

RESEARCH ARTICLE

Intrusion Detection for Wireless Sensor Network Using Particle Swarm Optimization Based Explainable Ensemble Machine Learning Approach

SHAIKH AFNAN BIRAHIM¹, AVIJIT PAUL², FAHMIDA RAHMAN³, (Member, IEEE),
YAMINA ISLAM³, TONMOY ROY⁴, MOHAMMAD ASIF HASAN², FARIHA HAQUE⁵,
AND MUHAMMAD E. H. CHOWDHURY⁵, (Senior Member, IEEE)

¹School of Computer Science and Engineering, University of Glasgow, G12 8QQ Glasgow, U.K.

²Department of Electronics and Telecommunication Engineering, Rajshahi University of Engineering and Technology, Rajshahi 6204, Bangladesh

³Department of Computer Science and Engineering, International Islamic University Chittagong, Chittagong 4318, Bangladesh

⁴Department of Data Analytics and Information Systems, Utah State University, Logan, UT 84322, USA

⁵Department of Electrical Engineering, Qatar University, Doha, Qatar

Corresponding author: Muhammad E. H. Chowdhury (mchowdhury@qu.edu.qa)

ABSTRACT Wireless Sensor Networks (WSN) play a pivotal role in various domains, including monitoring, security, and data transmission. However, their susceptibility to intrusions poses a significant challenge. This paper proposes a novel Intrusion Detection System (IDS) leveraging Particle Swarm Optimization (PSO) and an ensemble machine learning approach combining Random Forest (RF), Decision Tree (DT), and K-Nearest Neighbors (KNN) models to enhance the accuracy and reliability of intrusion detection in WSNs. The system addresses key challenges such as the imbalanced nature of datasets and the evolving complexity of network attacks. By incorporating Synthetic Minority Oversampling Technique Tomek (SMOTE-Tomek) techniques to balance the dataset and employing explainable AI methods such as Local Interpretable Model-agnostic Explanations (LIME) and SHapley Additive exPlanations (SHAP), the proposed model achieves significant improvements in detection accuracy, precision, recall, and F1 score while providing clear, interpretable results. Extensive experimentation on WSN-DS dataset demonstrates the system's efficacy, achieving an accuracy of 99.73%, with precision, recall, and F1 score values of 99.72% each, outperforming existing approaches. This work offers a robust, scalable solution for securing WSNs, contributing to both academic research and practical applications.

INDEX TERMS Intrusion detection system, wireless sensor networks, particle swarm optimization, ensemble machine learning, explainable AI, streamlit web application.

I. INTRODUCTION

Wireless Sensor Networks (WSN) are a key component in the field of wireless communication. These networks consist of sensor nodes arranged in various topologies such as star, tree, or mesh, which enable data sensing, transmission, and flow within the network. The primary functions of these sensor nodes include sensing, processing, computation, and

communication [1]. WSNs are widely employed for monitoring, security, and tracking purposes, offering a cost-effective solution with minimal energy consumption for wireless traffic communication. However, WSNs are vulnerable to unauthorized access by intruders, both internal and external, making it crucial to implement security measures to safeguard the network and ensure its integrity [2].

In complex and dynamic application scenarios, WSN nodes encounter several challenges: (1) limited computing power and storage capacity of individual nodes; (2) weak

The associate editor coordinating the review of this manuscript and approving it for publication was Stefano Scanzio¹.

communication capabilities between nodes; (3) operation within complex physical environments; and (4) mobility of certain nodes, leading to a dynamic and unpredictable network topology. As a result, WSN sensors tend to have lower security, making them more vulnerable to network attacks, with security issues becoming progressively more critical [3].

An Intrusion Detection System (IDS) is a software or hardware solution designed to detect and prevent unauthorized access to computer systems or networks [4]. IDS can be categorized into two types: host-based, which is integrated into a host device to monitor processes and user activity locally for intrusion detection, and network-based, which is the more common type. Network-based IDS operates across a network, monitoring traffic flow for potential intrusions in a distributed manner [5]. IDS classifies the Aegean Wi-Fi Intrusion Dataset (AWID) into four primary categories based on network behavior: Normal, Flooding, Injection, and Impersonation. Intrusion detection in IDS can be divided into four methods:

Anomaly-based IDS, which is effective for detecting unknown attacks by identifying abnormal behavior; Misuse-based (or signature-based) IDS, which detects known attacks by matching them with predefined signatures; Specification-based IDS, which monitors network components like routing tables, nodes, and protocols based on rules and thresholds to identify irregularities; Hybrid-based IDS, which combines anomaly, signature, and specification-based techniques. Wireless networks have surpassed wired networks in terms of flexibility, accessibility, mobility, and the lack of need for additional infrastructure [6]. Wireless Intrusion Detection Systems (WIDS) are specialized sensors designed to monitor and protect Wireless Local Area Networks (WLAN) by continuously scanning all operating frequencies within WLAN to detect and respond to intrusions [11]. The design goals of WSN are centered on improving data quality, reducing response time, enhancing network security, and extending network lifetime. Anomaly detection is critical for achieving these objectives, as abnormal data inevitably arises due to various uncontrollable factors. Such data can increase network traffic, delay, and waste limited energy resources [10], while also compromising subsequent data analysis and processing. Hence, timely and effective detection of anomalies is essential [7]. Despite the development of some solutions, challenges remain in terms of detection accuracy and speed. Leveraging artificial intelligence (AI) presents a promising avenue for overcoming these technical limitations. One significant challenge in WSN intrusion detection is the imbalanced nature of datasets, where normal instances vastly outnumber intrusion instances, negatively affecting detection accuracy. Moreover, WSNs often consist of a large number of sensor nodes, which leads to increased computational complexity and scalability issues [8]. High false positive rates and inadequate adaptability to evolving attack techniques further complicate intrusion detection [9]. To address these challenges, we propose a model that

combines Machine Learning (ML) techniques with the Synthetic Minority Oversampling Technique and Tomek Links (SMOTE-Tomek). This approach synthesizes minority intrusion instances while removing Tomek links, thereby balancing the dataset and improving detection accuracy and reliability. Our model represents a significant improvement over existing methods, providing an optimal solution for WSN intrusion detection.

The primary contributions of this study are as follows:

- 1) Developed a cutting-edge Intrusion Detection System (IDS) for Wireless Sensor Networks (WSN) that leverages Particle Swarm Optimization (PSO) for hyperparameter tuning in combination with an ensemble machine learning approach, significantly enhancing detection accuracy, adaptability, and model performance by efficiently optimizing parameter settings for better accuracy and generalization.
- 2) Implemented the SMOTE-Tomek technique to overcome the significant challenge of imbalanced datasets, leading to improved detection of minority attack classes in WSNs.
- 3) Incorporated state-of-the-art explainability tools such as LIME(Local Interpretable Model-agnostic Explanations) and SHAP (SHapley Additive exPlanations) to provide both local and global interpretations of the model's decisions, enhancing transparency and trust in its predictions.
- 4) Developed an interactive Streamlit web application to visualize the model's predictions and demonstrate its performance across different attack types and normal conditions in WSNs, making the system accessible and easy to interpret for users.
- 5) Utilized a robust ensemble of ML models including Random Forest (RF), Decision Tree (DT), and K-Nearest Neighbors (KNN), with soft voting to maximize the strength of multiple classifiers and deliver more reliable intrusion detection.

II. LITERATURE REVIEW

The topic of safety in wireless sensor networks has attracted a lot of interest lately. Various ML-based algorithms have been developed to stop network attacks or compromises. This section discusses a few of these models.

Dharini et al. [12] created their dataset using MATLAB with the main goal of figuring out how many characteristics would maximize model performance. Their findings indicated that peak performance was achieved using 14 features, resulting in an accuracy of 99.16% with both the LightGBM and XGBoost models, and 99.8% with the bagging method. Their study further demonstrated that reducing the number of features led to a decline in model performance, as models with only 5 or 3 features performed significantly worse. Notably, the LightGBM model was recognized for being the fastest, owing to its distinctive capabilities.

Tan et al. [13] emphasized how dataset imbalance affects model performance and how to fix it by balancing the

dataset using the Synthetic Minority Oversampling Technique (SMOTE). Initially, they used the random forest technique and got an accuracy of 92.39%. After applying SMOTE, the accuracy improved slightly to 92.57%, underscoring the importance of dataset balancing. Although the authors experimented with other techniques, classification performance remained a limitation for those models. The simplicity of the SMOTE-based model, however, contributed to better generalization and made it a practical solution for improving model accuracy.

Abhale and Reddy [14] conducted a comparative examination of various deep learning-based intrusion detection systems and presented updated methods, leading to performance improvements. In particular, the accuracy of the CNN-based algorithm went from 99.39% to 99.42%. Other models that shown notable improvements were those that were based on RNN, LSTM, and GRU. The authors emphasized that while deep learning is generally applied to large and complex datasets, its key advantage in this context is the reduction in human effort and improved feature correlation.

An IDS framework combining Cuckoo Search Greedy Optimization (CSGO) and Likelihood Support Vector Machine (LSVM) models was proposed by Hemanand et al. [15]. The system's accuracy and efficiency were improved by the LSVM component. Using popular network datasets like NSL-KDD and UNSW-NB15, the model was verified and was able to detect network assaults with a peak accuracy of 99.65%.

A deep learning-based intrusion prevention model was proposed by Chandre et al. [16], who achieved 97% accuracy through laborious computations utilizing a CNN-based methodology. The model established specific rules for each class, resulting in improved class separation. The research concluded that the model's performance remained consistent across datasets of varying sizes, suggesting that it had effectively learned features, making it adaptable and robust regardless of dataset size.

Without aiming to define particular attack types, Singh et al. [17] created a fuzzy rule-based method to divide network attacks into three groups. This approach resulted in improved predictive performance and reduced computational requirements, achieving an accuracy of 98.29% in preventing intrusions. The model focuses on intrusion prevention rather than anomaly classification, leaving approximately 1.17% of anomalies undetected after they have already entered the network. However, the model effectively addresses the intrusion prevention problem, offering a cost-efficient solution with strong performance.

A two-stage intrusion detection system based on deep neural networks was proposed by Gowdhaman and Dhanapal [18]. The model employed a cross-correlation approach in the first step to choose the best features, and then in the second stage, it applied a deep neural network to identify intrusions. With an accuracy of 95.53%, the system outperformed a number of well-known models. Nevertheless, other deep learning-based intrusion detection systems have

shown higher performance in spite of this one. This model was deemed too inefficient for general use due to the inherent difficulties and constraints of deep learning.

An intelligent hybrid model was proposed by Mohamed Behiry and Aly [19], which combined the K-means clustering model enhanced with information gain (KMC-IG) for feature extraction with feature reduction techniques like Singular Value Decomposition (SVD) and Principal Component Analysis (PCA) to reduce the number of features. Several datasets, including NSL-KDD, UNSW-NB 15, and CICIDS 2017, were used to validate the model. The main driving force for the entire study was feature reduction, which improved the model's performance from 98.9% to 99.7% in accuracy.

Notable results were obtained with the hybrid ML models for intrusion detection proposed by Gebremariam et al. [20]. Their work demonstrated a 99.80% accuracy on the NSL-KDD dataset using a hybrid Random Forest and Extreme Gradient Boost (RF-XGB) model, and a perfect 100% accuracy on the UNSW-NB15 and CICIDS2017 datasets with a hybrid Cluster Labeling K-Means (CLK-M) model. The overall accuracy across all hybrid models was 99.46%. The model is reliable, but the authors note that it does not completely remove the possibility of assaults on Internet of Things devices.

The fundamental goal of Elsadig [21] model was to achieve faster processing times without sacrificing competitive accuracy. The model achieved a detection accuracy of 99.5% and outperformed popular classifiers like Random Forest (RF), Extreme Gradient Boosting (XGBoost), and k-Nearest Neighbor (KNN) by lowering processing time to just 9.7%, 13%, and 2% of their respective times. The model was trained on a single dataset, which the author points out as a significant restriction. She suggests that using many datasets could result in a more effective and versatile model that is better suited for a range of circumstances.

In the context of WSNs, Karthikeyan et al. [22] suggested an intrusion detection system based on the firefly algorithm, examining the algorithm's potential for security-focused optimization methods. The firefly approach specifically used a support vector machine (SVM) model in conjunction with a grey wolf optimizer (GWO) for intrusion detection. At its peak accuracy of 99.34%, the model was able to utilize the firefly method.

A model specially developed for denial-of-service assaults was proposed by Liu et al. [23], and it was based on the k-nearest neighbor algorithm and the arithmetic optimization technique. In order to optimize the model's accuracy, population communication was improved through the use of a parallel method, and the optimization was adjusted using the Lévy flight strategy. These methods enabled the model to gain an approximate accuracy rate of 99%.

Binary Chimp Optimization Algorithm with ML-based Intrusion Detection (BCOA-MLID) approach was presented by Aljebreen et al. [24]. The BCOA was created with the best characteristics chosen in mind to increase effectiveness. With

a maximum accuracy of 99.36%, the model outperformed other widely-used models.

Nguyen et al. [25] introduced a unique strategy called Genetic Sacrificial Whale Optimization (GSWO) to address and overcome existing methods of intrusion detection. In addition to the efficient quantization of the GSWO algorithm, classification was performed using a CatBoost model. In the WSN-DS, WSNBFSF, NSL-KDD, and CICIDS2017 datasets, the GSWO-CatBoost approach has above 90% accuracy and has been reported to be faster than deep learning methods.

An intrusion detection system (IDS) tailored for software-defined 5G networks was proposed by Bocu et al. [26]. The solution leverages the scalability and adaptability of software-defined networks (SDN) for efficient implementation by integrating artificial intelligence models to detect intrusions that were previously undetected. The system's accuracy falls short of some of the most advanced models in the field, despite its practical usefulness for real-world deployment in 5G contexts.

Despite significant advancements in IDS for WSN, several research gaps remain. First, many existing models are limited by dataset imbalances, which affect detection accuracy and lead to high false-positive rates. While techniques like SMOTE improve class balance, the performance gains are often marginal. Second, many approaches, including optimization algorithms like PSO and Firefly, have demonstrated high accuracy but tend to suffer from computational inefficiency, scalability issues, or limited generalization across diverse datasets. Furthermore, while deep learning models have shown promise, their high complexity and computational demands restrict their practicality for real-time applications. Lastly, intrusion prevention, especially in dynamic and large-scale WSN environments, remains underexplored, with a focus predominantly on detection rather than preemptive measures to block evolving attacks. Addressing these gaps requires the development of more efficient, scalable, and adaptive models capable of handling dynamic network conditions and real-world attack scenarios.

III. METHODOLOGY

In this study, a comprehensive approach to intrusion detection in WSN is presented, utilizing a robust ML model that has been optimized for performance and interpretability through advanced data processing and ensemble techniques. The dataset, known as WSN-DS, contains diverse real-world network scenarios and attack conditions. Preprocessing of the dataset involved converting categorical variables into numerical forms for compatibility with the machine learning model. To address the class imbalance, SMOTE was applied to balance the dataset. The data was split into 80% for training and 20% for testing. Hyperparameter tuning was performed using PSO to enhance model performance, as it proved to be highly efficient and emerged as the most effective technique compared to other optimization techniques. Various machine learning methods were employed in the process of finding

best outcome, and at the end combining different algorithms proved to be effective. Thus, an ensemble model comprising RF, DT, and KNN algorithms was employed, with its effectiveness evaluated using 5-fold cross-validation. Metrics such as accuracy, precision, recall, and F1 score were used to assess the model's performance. SHAP and LIME methods were incorporated to provide global and local explanations of the model's behavior.

A. DATASET DESCRIPTION

A dataset called WSN-DS by Almomani et al. [41] is intended for the identification of DoS assaults, comprising 374,661 records, with 9% identified as DoS attacks. The LEACH system, a popular hierarchical routing system in Wireless Sensor Networks (WSNs), was used to curate this dataset, which covers typical network behaviors and four different kinds of DoS assaults (Grayhole, Blackhole, TDMA, and Flooding). Network Simulator 2 (NS-2) was used to gather data, and after processing, 23 features were produced, adding value to the dataset. Researchers looking to create strategies for mitigating DoS attacks and improving network security will find this dataset to be a useful resource [27].

B. DATA PREPROCESSING

Data preprocessing, which involves cleaning and transforming raw data to ensure its quality and suitability for analysis, is an essential stage in the construction of models. This process is vital for the model's effectiveness, particularly in achieving successful detection in Wireless Sensor Networks (WSN) and ensuring data compatibility with ML models. The subsequent section will detail the specific preprocessing steps applied to the dataset.

1) MANUAL FEATURE SELECTION

Manual feature selection is the identification of features for incorporation into a machine learning model based on domain expertise, intuition, or exploratory data analysis, rather than depending on automated techniques. This method depends on human discernment to identify and eliminate features considered unnecessary, redundant, or unhelpful for augmenting the model's prediction efficacy. In the WSN dataset, the feature labeled 'id' was considered extraneous for attack detection, as it failed to demonstrate any patterns linked to assault scenarios. As a result, this feature was eliminated during the preprocessing stage. Omitting this feature improves the model's effectiveness by decreasing the amount of features it must analyze, enabling it to concentrate exclusively on the most pertinent data.

2) LABEL ENCODING

In ML, label encoding is a technique that transforms category input into a numerical representation that is appropriate for algorithmic processing. When dealing with categorical characteristics that have a limited number of different values, this strategy is quite helpful [28]. The target variable in the

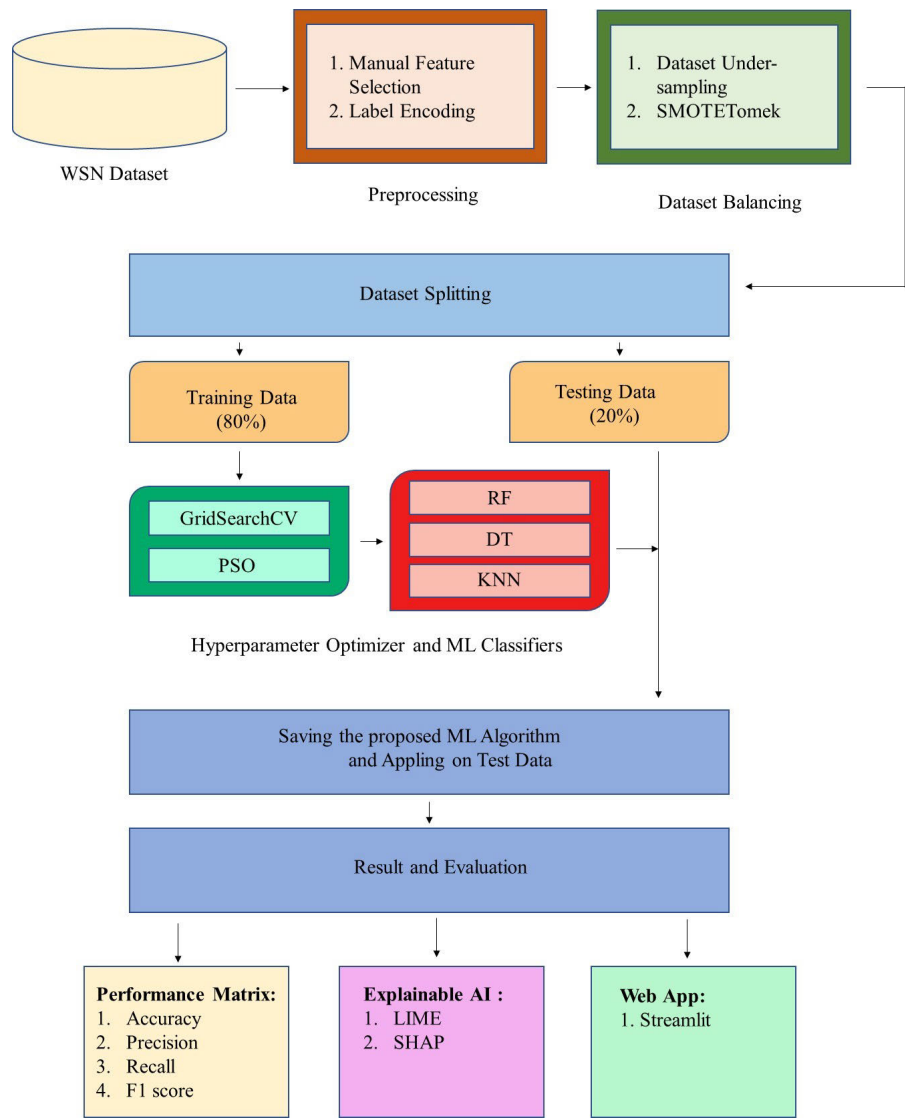


Figure 1. Overall Workflow of the proposed model.

WSN dataset was categorical, making it inappropriate to employ ML methods directly in this situation. Label encoding was used to address this, giving each class a distinct discrete numerical value. Five classifications were included in the dataset: Flooding, TDMA, Grayhole, Blackhole, and Normal. The table 2 displays the label encoding for these classes.

The process of label encoding assigned a unique discrete numerical value to each target class, thereby making them suitable for the implementation of ML algorithms.

3) STANDARDIZATION

Standardization, often referred to as z-score normalization, is a preprocessing method used in statistics and ML to change a dataset’s features so that their means and standard deviations are both one. This method helps to lessen the negative effects on the performance of some algorithms, therefore it is especially useful when working with data that

has different scales [29]. The formula for standardization is as follows:

$$z = \frac{x - \mu}{\sigma} \tag{1}$$

here, μ is the feature mean, z is a standardized value, and x is the initial feature value. The feature’s standard deviation is σ . Given the varied range of data inside the WSN-DS dataset, scaling becomes vital for assuring effective analysis and model performance. By adopting standardization, all features are brought to the same scale, ensuring that each feature contributes equally to the model and preventing any single feature from overwhelming others. In the end, this procedure improves the accuracy and performance of the ML algorithms used on the dataset by preserving its balance and integrity.

The pretreatment methods used here guarantee that the data is optimally ready for the ML algorithm to run quickly.

Table 1. Feature description.

Feature symbol	Description
id	A unique ID to distinguish the sensor node in any round and at any stage.
Time	The current simulation time of the node.
Is_CH	A flag to distinguish whether the node is cluster head with value 1 or normal node with value 0.
who_CH	The ID of the cluster head in the current round.
Dist_To_CH	The distance between the node and its cluster head in the current round.
ADV_S	The number of advertise cluster head's broadcast messages sent to the nodes.
ADV_R	The number of advertise cluster head messages received from cluster heads.
JOIN_S	The number of join request messages sent by the nodes to the cluster head.
JOIN_R	The number of join request messages received by the cluster head from the nodes.
SCH_S	The number of advertise Time Division Multiple Access (TDMA) schedule broadcast messages sent to the nodes.
SCH_R	The number of Time Division Multiple Access (TDMA) schedule messages received from cluster heads.
Rank	The order of this node within the Time Division Multiple Access (TDMA) schedule.
DATA_S	The number of data packets sent from a sensor to its cluster head.
DATA_R	The number of data packets received from cluster head.
Data_Sent_To_BS	The number of data packets sent to the base station.
dist_CH_To_BS	The distance between the cluster head and the base station.
send_code	The cluster sending code.
Expanded Energy	The amount of energy consumed in the previous round .
Attack type	Attack Type: It is a class of five possible values, namely, Blackhole, Grayhole, Flooding, Time Division Multiple Access (TDMA) which also refers to Scheduling and Normal.

Table 2. Label encoding used in the proposed method.

Class	Label
Black hole	0
Flooding	1
Grayhole	2
Normal	3
TDMA	4

Handled feature selection reduces superfluous computing load by eliminating unimportant features. The technique performs better when the data is scaled to a consistent range by standardization. Furthermore, label encoding converts the target class into a numerical format, which makes it easier for the ML models to process.

C. DATA BALANCING

A dataset's class distribution must be adjusted for data balancing in order to guarantee that each class is represented

proportionately or evenly. This is especially crucial for classification tasks because an imbalance in class can lead to the model being biased in favor of the majority class, which would make the minority class perform poorly in predictions. The normal class, or scenarios without anomalies, has a far larger number of records in the WSN dataset than do the attack scenarios. The assault records form only 9% of the overall dataset. When there is an imbalance, the model may forecast classes inconsistently, doing well for the majority class but poorly for the minority class. Consequently, if the data continues to be uneven, performance measurements will be less trustworthy.

The SMOTETomek methodology, a hybrid data balancing method, is used to address the issue. To efficiently balance the dataset, SMOTETomek combines Tomek Links and SMOTE [33].

Using an oversampling approach called SMOTE, artificial samples are produced for the minority class. SMOTE helps to build a more broad decision boundary by creating new samples by interpolating between existing ones rather than replicating current minority samples [30], [31].

Tomek Links are groups of samples from various classes that are closest to one another. These couples are frequently found close to the class boundary and are regarded as ambiguously classed. The dataset is cleaned by removing these pairs, which also clarifies the decision border between classes [32].

In concert, SMOTETomek detects and eliminates Tomek linkages from the dataset after oversampling the minority class. This procedure produces a cleaner dataset by removing erratic and noisy records. By combining these methods, the performance of the model is enhanced by the creation of a more precise and comprehensive decision boundary.

Random Oversampling and Cluster-based Sampling exhibit limitations that reduce their effectiveness compared to SMOTE-Tomek. Random Oversampling often results in the repetition of existing data, which does not contribute new information and can increase the risk of overfitting. Although Cluster-based Sampling offers an alternative, it is computationally demanding and may not achieve the same performance as SMOTE-Tomek. The SMOTE-Tomek method combines the advantages of generating synthetic samples and eliminating noisy instances through Tomek Links, thereby producing a more balanced dataset with clearer decision boundaries.

In summary, balancing the dataset is crucial for enhancing classification performance in ML models. SMOTETomek effectively addresses the imbalance issue, making the dataset balanced and more suitable for ML algorithms.

D. MODELS AND ALGORITHMS

1) ENSEMBLE LEARNING

An ML method known as “ensemble learning” involves training and combining several models to address a specific issue. By utilizing the strengths of several models and reducing their individual flaws, ensemble learning aims to

improve the predictive model's performance and robustness. The Voting Classifier is a prominent ensemble learning technique that combines predictions from numerous models to produce a final prediction. By combining the predictions of multiple reliable models, this method can improve performance and resilience.

There are two sorts of Voting Classifiers: hard and soft.

- 1) **Hard Voting Classifier:** The majority class label that each individual model predicts serves as the basis for the final prediction in this strategy. Every model has one vote, and the class with the most number of votes becomes the final forecast.
- 2) **Soft Voting Classifier:** The average of the anticipated probabilities serves as the basis for the final forecast in this procedure. The ultimate forecast is made for the class with the highest average likelihood. Soft voting often performs better when the basis classifiers are well-calibrated.

In this proposed work, soft voting classifier is used to aggregate the predictions of different base models. The equation for the soft voting classifier is given below:

$$g(j) = \arg \max_j P(j|x) = \arg \max_j \left(\frac{1}{N} \sum_{i=1}^N P(j|x_i) \right) \quad (2)$$

where, $P(j|x_i)$ represents the predicted probability of class j by the k -th classifier for instance x . N is the number of classifiers in the ensemble. $P(j|x)$ is the average predicted probability of class j over all classifiers. $g(j)$ denotes the final predicted class label for instance x .

The ensemble learning technique is an essential part of the proposed work because it has been shown to be very effective in producing more consistent and reliable predictions. Ensemble learning improves overall model performance and robustness by utilizing the combined strengths of numerous models, which greatly increases the final predictions' accuracy and reliability.

2) PARTICLE SWARM OPTIMIZATION (PSO)

The practice of tweaking an ML model's hyperparameters in order to improve its performance is known as hyperparameter tuning. The parameters that control the learning process are known as hyperparameters, and they are established before training. The accuracy of the model can be greatly increased by carefully adjusting these hyperparameters. By determining the ideal parameters, this procedure contributes to the model's increased robustness. The suggested approach makes use of Particle Swarm Optimization (PSO), an optimization method inspired by nature, for this objective.

A computational method called PSO is used to improve a candidate solution step-by-step based on a predetermined quality metric. It mimics the social behavior of fish in a school or birds in a flock to address optimization problems. The suggested method begins with an optimizer choosing a particle, which stands for a possible answer in the search space. Next, these particles' locations and velocities are set

to zero. After initialization, the global and personal best positions are used to update the velocity and position. The following equations can be used to update the position and velocity:

Velocity update equation:

$$v_i(t+1) = v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t)) \quad (3)$$

where, $v_i(t+1)$ is the updated velocity of particle i at time $t+1$. $v_i(t)$ is the velocity of particle i at time t . c_1 is the cognitive acceleration coefficient. c_2 is the social acceleration coefficient. r_1 and r_2 are random numbers between 0 and 1. $pbest_i$ is the personal best position of particle i . $gbest$ is the global best position. Position update equation:

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (4)$$

where, $x_i(t+1)$ is the updated position of the particle i at time $t+1$. $x_i(t)$ is the position of the particle i at time t . $v_i(t+1)$ is the updated velocity of particle i at time $t+1$.

Following the update phase, the optimizer uses cross-validated accuracy to assess each particle's fitness, which measures the performance of the hyperparameters represented by the particle. Each particle in the swarm corresponds to a specific set of hyperparameters. These hyperparameters are evaluated by training model and calculating its cross-validated accuracy to ensure the solution's robustness and generalization to unseen data. Iteratively, the particles update their positions and velocities based on their personal best positions and the global best position of the swarm, ensuring a balance between exploring new regions of the search space and exploiting promising solutions. The personal best captures the particle's most successful hyperparameter configuration, while the global best represents the highest-performing configuration across the swarm. By leveraging this dual influence and incorporating stochasticity to avoid local optima, the particles systematically refine their positions. Through this iterative process, the optimizer ensures an efficient search of the hyperparameter space, ultimately converging on an optimal set of parameters that enhance the model's accuracy and generalization.

In conclusion, PSO is a potent tool for fine-tuning hyperparameters. Through simultaneous and global search processes, it improves the effectiveness and efficiency of the tuning process, making the optimization more adaptive. As a result, the model performs better since the ideal hyperparameters are systematically found, guaranteeing the model's accuracy and stability.

3) DECISION TREES (DT)

For problems involving regression and classification, a supervised learning technique called a decision tree is employed. It creates a decision tree by dividing the data into subgroups according to feature values. Every leaf node represents a class label or a continuous value, every branch reflects the decision's result, and every internal node represents a

choice based on a feature. In order to divide the data into branches that lead to more nodes or leaf nodes, the algorithm chooses the best characteristic at each node. Recursively repeating this process continues until a halting condition is satisfied. Although decision trees are simple to understand and intuitive, they are susceptible to overfitting, particularly when dealing with noisy and deep trees.

4) RANDOM FOREST (RF)

One popular ensemble learning method for both classification and regression applications is Random Forest. During the training phase, this approach builds many decision trees, and the final output is based on the mean forecast of each individual tree or the mode of the classes. Random Forest improves prediction accuracy and stability by combining the predictions from several trees. The integration of randomness in data sampling and feature selection contributes to the construction of different trees, hence limiting the risk of overfitting. It also offers estimates of feature relevance, which helps with feature selection and can handle huge, high-dimensional datasets. All things considered, Random Forest is a strong and flexible ensemble technique that is well-liked across a range of ML applications due to its excellent accuracy and resilience.

5) K-NEAREST NEIGHBORS (KNN)

KNN is a straightforward, non-parametric, and lazy learning method that predicts the result based on the k-nearest data points in the feature space. It is utilized for both regression and classification tasks. The algorithm runs by storing all accessible data points and, for all new input, calculating the distance to all training data points, sorting these distances, selecting the k closest points, and utilizing them for prediction. It uses neighborly majority voting for categorization and neighbor average computation of values for regression. KNN's main benefits are its simplicity, ease of use, adaptability, and short training period. Nevertheless, it can be memory- and computationally-intensive, and it is susceptible to data scaling and unimportant aspects. All things considered, KNN is a useful technique for a variety of applications, especially when the data quantity is reasonable and the feature significance is obvious.

E. EXPLAINABLE AI (XAI)

The term Explainable AI (XAI) is a collection of approaches and procedures designed to increase human comprehension and transparency of artificial intelligence (AI) system outputs. Ensuring that users can comprehend and interpret AI models, like ensemble approaches, is the aim of explainable AI. Two frameworks have been used to explain the planned task.

1) LOCAL INTERPRETABLE MODEL-AGNOSTIC EXPLANATIONS(LIME)

The method known as LIME (Local Interpretable Model-agnostic Explanations) is used to analyze and comprehend

the predictions made by intricate ML models. By using an interpretable model to approximate it locally, it provides an explanation for any black-box model's predictions. LIME offers many explainable models, each specific to each input point, that closely mimic the behavior of the original model. The explainable model guarantees that its predictions will be quite close to the black-box model around this input point [36]. LIME is used in this proposed effort to better understand the behavior of the model and to determine the rationale behind particular predictions.

2) SHAPLEY ADDITIVE EXPLANATIONS(SHAP)

The SHAP framework clarifies how various features contribute to a model's output, making it possible to evaluate predictions made by ML models. SHAP values, which have their roots in cooperative game theory, offer a way to comprehend how each attribute affects the prediction for a certain instance [34], [35]. It can be difficult to describe a model's reliability when it is complex. Although accuracy is a useful performance metric, it doesn't explain how the model learns or generates predictions. In order to solve this, SHAP provides visualization tools that illustrate how each feature in a dataset contributes to the prediction, explaining the results of any ML method. There are several SHAP explainers available; the explainers used in this proposed work are Tree and Kernel. The results of this proposed study are explained via a series of plots provided by SHAP, which allow for an in-depth examination of the properties of the model.

F. STREAMLIT

Streamlit is an application framework that makes it simple to create online applications for ML. It has excellent compatibility with the main Python packages that are frequently used for data processing and visualization in ML. Because of this interoperability, developers may easily create visually appealing and interactive applications to present their ML models and outcomes [37].

IV. RESULTS ANALYSIS & DISCUSSION

This section serves as the core of our research, unveiling the results and findings of our work. Multiple experiments were conducted to identify the most suitable model. The first goal is to compare the models' performance using various performance metrics. The second challenge is to use explainable AI approaches to explain the proposed model and its effectiveness in comparison to other models. The part is finally concluded with a web application that shows the model in action.

A. ABLATION STUDY

In this ablation study section, various adjustments and combinations of models and optimization techniques were tested to identify the best possible outcome. By evaluating different configurations, the study aimed to determine the most effective approach for achieving optimal model

Table 3. Model with tuned hyperparameter names.

Model	Hyperparameter	Value
Decision Tree	max_depth	50
	min_samples_split	8
	min_samples_leaf	1
Random Forest	n_estimators	55
	max_depth	50
	min_samples_split	2
	min_samples_leaf	1
K Nearest Neighbor	n_neighbors	3
	weights	distance
	algorithm	brute

performance. The adjustments were made on the following components:

- **Ensemble Learning:** The soft vote classifier approach of ensemble learning was used to assemble different independent models in different combinations.
- **Particle Swarm Optimization:** The importance of a nature-inspired optimization technique like PSO was assessed by incorporating it into and removing it from the task of hyperparameter tuning.
- **GridSearchCV:** The capabilities of GridSearchCV as a hyperparameter tuner were examined by comparing models with and without its use.

The ablation study is essential to choosing the optimal model. Finding the best mix of model designs, optimization strategies, and preprocessing stages requires systematic testing and comparing of different parts, configurations, and methodologies. By ensuring that every component's contribution to the overall performance is understood, this in-depth research leads to the creation of the most reliable and accurate model.

B. HYPERPARAMETER SELECTION

Hyperparameter selection is a critical step in the machine learning model development process, as it directly influences the model's performance and generalizability. Properly selecting hyperparameters helps optimize the model, balancing bias and variance to achieve better predictive accuracy.

Table 3 presents the optimized hyperparameters for each model, which were used to form the ensemble that constitutes the proposed model. For the Decision Tree (DT), 'max_depth' refers to the maximum allowable depth of the tree, 'min_samples_split' denotes the minimum number of samples required to split an internal node, and 'min_samples_leaf' specifies the minimum number of samples needed at a leaf node. In the case of the Random Forest (RF), 'n_estimators' represents the number of trees in the forest, while the remaining parameters are identical to those of the Decision Tree. For the K-Nearest Neighbors (KNN) classifier, 'n_neighbors' defines the number of neighbors to consider for classification, 'weights' determines how the neighbors are weighted, and 'algorithm' specifies the algorithm used to identify the nearest neighbors.

C. PERFORMANCE METRICS

Performance indicators are essential for evaluating a machine-learning model's efficacy. They are especially important for assessing the results of categorization models. These metrics provide insights into how well the model produces predictions and help the comparison of alternative models or changes to the existing one. The performance matrices used to analyze the proposed work are discussed below:

1) ACCURACY

The most widely used performance indicator for determining the percentage of accurate predictions produced by the ML model is accuracy. It can be calculated as:

$$Accuracy = \frac{TruePositives + TrueNegatives}{TotalNumberofPredictions} \quad (5)$$

2) PRECISION

The percentage of true positives among all positive predictions is measured by precision, which is often referred to as the positive predictive value. Greater accuracy in the model's positive classifications is ensured by fewer erroneous positive predictions, which are indicated by a higher precision value. Precision can be calculated as:

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives} \quad (6)$$

3) RECALL

Recall, which is sometimes referred to as sensitivity, quantifies the percentage of real positives that are true positive predictions. A high recall value guarantees a low false negative rate, meaning the majority of positive examples are accurately identified by the model. Recall can be calculated as:

$$Recall = \frac{TruePositives}{TruePositives + FalseNegatives} \quad (7)$$

4) F1 SCORE

The F1 score is a balanced statistic that considers both precision and recall, calculated as the harmonic mean of these two metrics. With a single statistic that represents the model's performance in terms of both false positives and false negatives, it is particularly helpful for balancing the trade-off between precision and recall. F1 score is calculated as:

$$F1score = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (8)$$

Certain performance measures can be quite helpful in certain situations. For example, whereas accuracy might be adequate in many circumstances, precision becomes essential when the cost of false positives is significant. However, recall is crucial when it comes to missing a real positive because it may be very expensive. When a balance between recall and precision is required, the F1 score is useful as it provides a thorough evaluation of the model's performance.

D. PERFORMANCE RESULTS

This section illustrates the performance of all the different models tested throughout this study work. Using SMOTE-Tomek, the dataset was balanced prior to testing each model. In all cases, the parameters were tuned using either GridSearchCV or PSO. For models where ensemble learning was applied, the soft voting method was utilized. Table 4 presents the results, including accuracy, precision, recall, and F1 score, computed using the weighted average method for all the different models tested to achieve the best possible outcome.

DT models achieved a maximum accuracy of 99.41% when using PSO for hyperparameter tuning. DT is considered to be well-suited for this research because of its popularity, ease of understanding and presentation, and ability to handle both numerical and categorical data with ease. Given the relatively large size of the dataset, DT's speed and low computational requirements made it an appealing choice. However, DTs can be biased if the dataset is imbalanced. This limitation was addressed by balancing the dataset during preprocessing. Additionally, since the dataset was substantial, using multiple DTs could enhance performance and robustness.

RF model's 99.68% accuracy on the WSN dataset shows how effective ensemble approaches can be at outperforming a single DT. You pointed out that one disadvantage of the RF is that it requires more computing power to handle massive datasets and achieve great accuracy. However, by balancing the dataset and using hyperparameter tuning techniques like PSO and GridSearchCV, you've maximized the model's potential, making it the top performer among the classifiers tested. Despite the challenges, the RF's robustness and slight accuracy improvement make it a compelling choice for your anomaly detection task.

The implementation of the KNN classifier in the research work yielded an accuracy of 98.97% on the WSN-DS dataset. While this accuracy is slightly lower compared to some tree-based models, KNN's ability to handle multiclass classification effectively adds value to the research. The potential for improved performance with feature reduction exists, but it may compromise the model's robustness. The inclusion of KNN diversifies the approach and contributes to the overall exploration of different classifiers in the study.

When comparing the performance of the base classifiers with two different hyperparameter tuning algorithms, it's evident that PSO is a more suitable choice than GridSearchCV. GridSearchCV, while thorough, performs an exhaustive search over a specified parameter grid, making it time-consuming and computationally intensive, especially with large datasets. On the other hand, PSO is more efficient with large datasets due to its probabilistic nature, requiring significantly less computation. This efficiency makes PSO the better alternative for hyperparameter tuning in this context.

The tree-based models performed well, so combining the two tree-based algorithms, RF and DT, seemed like a good strategy for achieving diverse outcomes. Although RF is

Table 4. Performance metrics analysis for different ML algorithms using PSO and GridSearchCV.

Algorithm	Accuracy	Precision	Recall	F1 Score
RF + GridSearchCV	99.68%	99.69%	99.68%	99.68%
RF + PSO	99.68%	99.68%	99.68%	99.68%
DT + GridSearchCV	99.4%	99.4%	99.4%	99.4%
DT + PSO	99.41%	99.41%	99.41%	99.41%
KNN + GridSearchCV	98.97%	98.98%	98.97%	98.96%
KNN + PSO	98.97%	98.98%	98.97%	98.97%
(RF+DT) + PSO	99.43%	99.43%	99.43%	99.43%
(RF+KNN) + PSO	99.56%	99.57%	99.57%	99.57%
(RF+KNN+XGB) + PSO	99.69%	99.69%	99.69%	99.69%
(RF+DT+KNN) + PSO	99.73%	99.72%	99.72%	99.72%

already an ensemble of DTs, combining them in a voting classifier still adds value by increasing the model's diversity. The ensemble of RF and DT achieved an accuracy of 99.43%, which is better than DT alone but slightly less than RF.

However, the model required further adaptation to handle complex decision boundaries and higher sensitivity. Therefore, by ensembling RF, DT, and KNN using a soft voting method, it was possible to overcome these limitations. This combined model, tuned with Particle Swarm Optimization (PSO), achieved an accuracy of 99.73%, making it the best model so far. The combination of these three classifiers results in a model that is more reliable and robust.

In the exploration of alternatives for DT within the ensemble framework—given that the best-performing model thus far incorporated two tree-based models—various classifiers were evaluated. Among these, the XGBoost (XGB) classifier, when combined with RF and KNN, and hyperparameters optimized using PSO, achieved a notable accuracy of 99.69%. While this performance was impressive, it did not surpass DT as the most effective model in the ensemble. The optimal hyperparameters for XGB obtained through PSO were: 'n_estimators' = 51, 'max_depth' = 9, and 'learning_rate' = 0.3. Based on these findings, RF, DT, and k-nearest neighbors (KNN) were selected as the models for the ensemble.

E. COMPARISON WITH PREVIOUS RESEARCH

The proposed method outperforms existing state-of-the-art (SOTA) works by addressing several key challenges

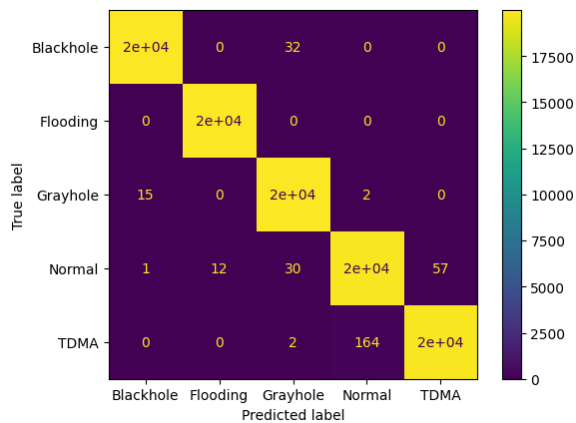


Figure 2. Confusion matrix of RF with GridSearchCV.

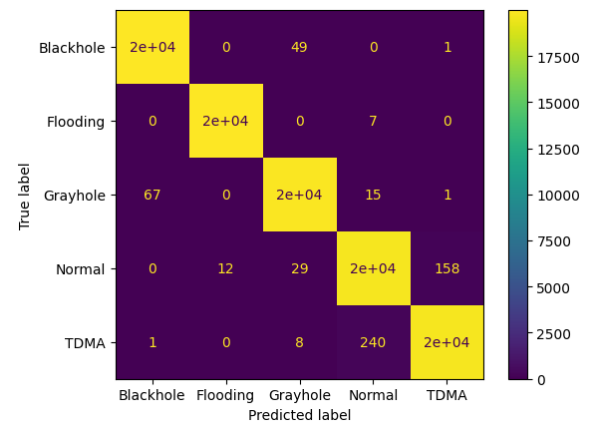


Figure 5. Confusion matrix of DT with PSO.

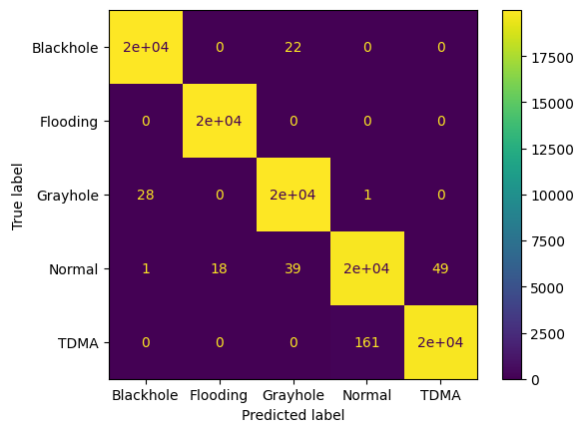


Figure 3. Confusion matrix of RF with PSO.

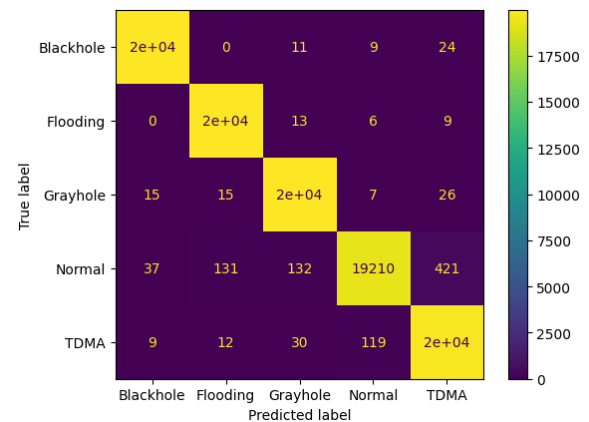


Figure 6. Confusion matrix of KNN with GridSearchCV.

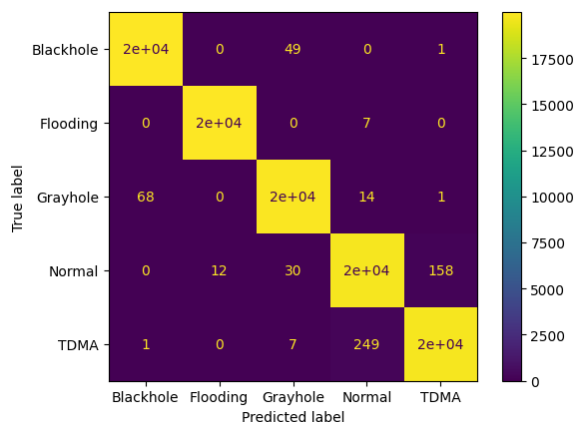


Figure 4. Confusion matrix of DT with GridSearchCV.

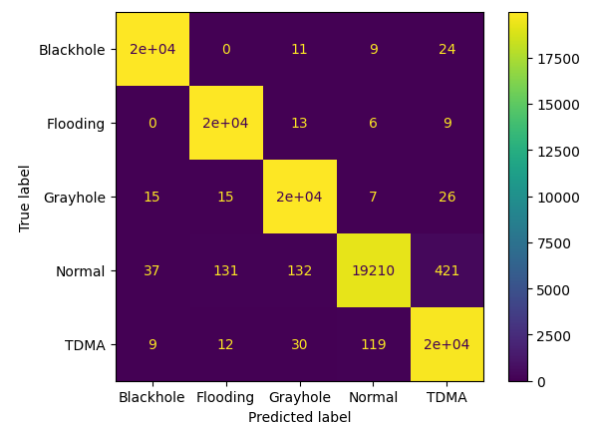


Figure 7. Confusion matrix of KNN with PSO.

that have limited the effectiveness of previous approaches. First, many models faced issues with dataset imbalance, affecting detection accuracy. While techniques like SMOTE were employed in prior studies, our approach leverages the SMOTE-Tomek method, which not only balances the dataset by oversampling the minority class but also removes noisy instances using Tomek links, improving both accuracy and generalization. Various methods have been implemented

in the past by different researchers to detect anomalies in wireless sensor networks. Table 5 highlights some of these remarkable works. While these methods were considered among the best during their respective times, they still had certain drawbacks.

Jeevaraj et al. [38] obtained a somewhat better outcome by using a variety of ML classifiers, including Naive Bayes, BayesNet, and OneR, to determine the best model. The IBK

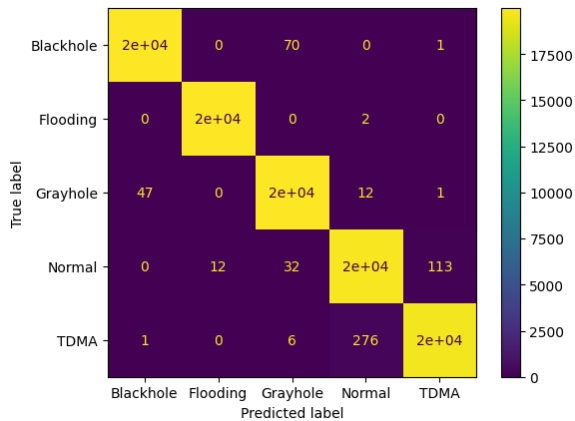


Figure 8. Confusion matrix of voting classifier (RF+DT) with PSO.

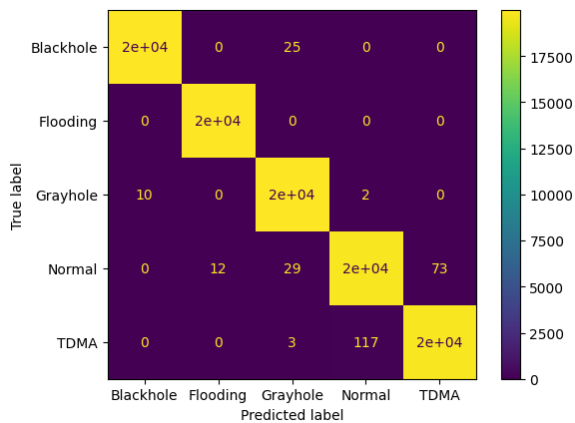


Figure 9. Confusion matrix of voting classifier (DT + RF + KNN) with PSO.

Table 5. Comparison of proposed model with previous research works.

Ref.	Approach	XAI	Accuracy	Precision	Recall	F1 Score
[16]	CNN	No	97%	83.6%	82.6%	82%
[1]	CNN + RNN	No	97.04%	98.79%	96.48%	96.86%
[25]	Catboost + GSWO	No	99.62%	97.39%	97.78%	97.47%
[38]	IBK	No	99.448%	99.4%	99.4%	99.4%
[39]	Naive Bayes	No	95.82%	96.8%	95.4%	-
[21]	DT+Gini feature selection	No	99.5%	-	-	-
Proposed	(RF+DT +KNN) + PSO	Yes	99.73%	99.72%	99.72%	99.72%

classifier, with an accuracy score of 99.448%, was the top performance among them. Sixty-six percent of the dataset was used for training, and the remaining fraction was used for validation and testing. However, a significant limitation of this research is the lack of preprocessing steps, despite the highly imbalanced nature of the WSN dataset. This omission may render the results less reliable. Nevertheless, the model's

high accuracy, achieved without extensive computational requirements, is commendable.

Salmi and Oughdir [1] and Chandre et al. [16] both adopted deep learning-based approaches, implementing Convolutional Neural Networks (CNN) for enhanced classification. In addition to CNN, one of the studies also experimented with Deep Neural Networks (DNN) and Recurrent Neural Networks (RNN). However, these models only managed to achieve around 97% accuracy. Moreover, deep learning models typically require extensive computational resources. Given that several ML models can achieve better performance with less computational effort, the necessity for these deep learning approaches becomes less compelling.

Jeevaraj et al. [39] proposed a different approach, focusing on feature selection rather than solely relying on classifiers. Their method involved iteratively removing less important features and re-evaluating the model's performance after each iteration. While this approach thoroughly explored feature importance, reducing the number of features did not significantly improve the model's performance. Naive Bayes was used as the classifier, and the model achieved its best accuracy score of 95.82% using the six most important features.

Elsadig [21] also emphasized feature selection as a strategy to achieve better results with reduced computational effort. In this study, the Gini feature selection method was employed alongside the Decision Tree classifier. The Gini method selected 16 out of 18 features from the dataset. However, given the highly imbalanced nature of the dataset, incorporating additional preprocessing steps could have enhanced the model's reliability and robustness for real-world applications. Despite the absence of such steps, the model still achieved a commendable accuracy of 99.5%.

The best-performing model from previous works was developed by Nguyen et al. [25], who used GSWO for hyperparameter tuning and feature selection, combined with the CatBoost classifier for classification tasks. The model demonstrated excellent performance, achieving an accuracy of 99.62%. However, since the dataset was imbalanced, additional steps to prevent the overfitting of specific classes were necessary. Moreover, the computational demands of the GSWO optimizer were quite high. Despite these challenges, the model's performance was outstanding. Additionally, XAI methods such as SHAP and LIME, which were not used in prior works, offer a distinct advantage in our model. These techniques allow us to provide interpretable insights into the model's decision-making process, giving both local and global explanations of feature importance. This transparency not only increases the trustworthiness of the model but also enables a deeper understanding of why certain predictions are made—an aspect often missing in black-box models like deep learning approaches used by other authors. Moreover, our approach integrates PSO for hyperparameter tuning, which enhances the model's performance by efficiently finding optimal parameters. By combining an ensemble of classifiers—Random Forest, Decision Tree, and KNN—with

Table 6. Actual values of the test sample chosen for the LIME explanation in the Figure 10.

Feature Names	Actual values
SCH_S	13.00
ADV_S	1.00
DATA_S	0.00
JOIN_S	0.00
JOIN_R	13.00
Data_Sent_To_BS	0.00
DATA_R	0.00
dist_CH_To_BS	0.00

soft voting and PSO tuning, our model achieved 99.73% accuracy, outperforming models such as the CNN-based approaches, which reached only around 97%. This demonstrates our method’s ability to handle complex WSN intrusion scenarios more effectively. By addressing the research gaps discussed in the literature, such as imbalanced datasets, high false positives, and the need for interpretability, the proposed model offers a robust, scalable, and explainable solution for WSN intrusion detection.

F. MODEL’S EXPLAINABILITY

In this research work, the use of Explainable AI (XAI) techniques like LIME and SHAP allowed for the determination of each feature’s importance and provided explanations for the impact of every feature on specific decisions through both local and global representations. Local representation refers to explaining the influence of individual features on a particular test set, accomplished using LIME, SHAP force plots, and SHAP waterfall plots. Global representation, on the other hand, explains the importance of features across a group of test sets. This was demonstrated using SHAP summary plots, SHAP summary plots with violin distributions, and force plots encompassing all test data.

1) EXPLAINABILITY OF THE OUTCOME USING LIME (LOCAL)
LIME plots provide a clear visual explanation of each feature’s positive and negative impact on decision-making, offering deeper insights into why a particular decision was made by the proposed model. Figure 10 presents the LIME plot for the top eight most impactful features, showing their influence on determining whether the network is normal or under attack at a specific instance. A horizontal bar that extends to the right and is colored green in the plot indicates a feature’s beneficial influence on the decision-making process. On the other hand, a red-colored bar that extends to the left indicates a negative influence. These bars’ lengths indicate how much of an impact each feature has, making it easy to see how much of a contribution each feature contributes to the model’s conclusion.

According to Figure 10 of the LIME explanation, the feature named “SCH_S” has the most significant impact on the decision-making process for determining whether the network is normal or under attack. Table 6 indicates that the value of “SCH_S” is 13.00. However, the LIME explanation

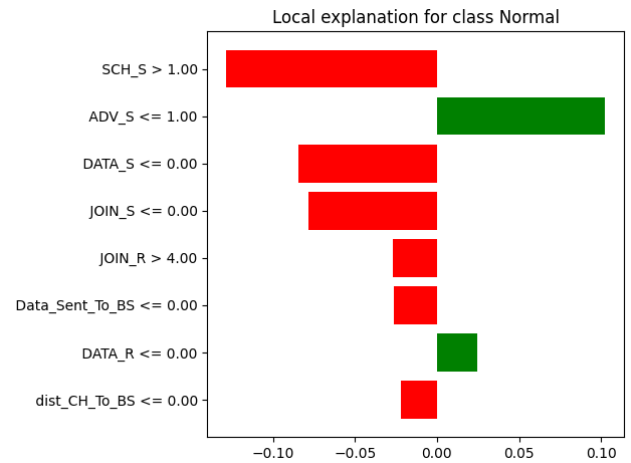


Figure 10. LIME explanation for normal class.

reveals that the model has set a threshold value for “SCH_S” at 1 or below for classifying a network state as normal. Since the value of “SCH_S” is considerably higher than the threshold, the model identifies this as an attack situation. Similarly, all features that fall within or outside the range set for normal classification contribute to either a positive or negative impact on the model’s decision. The magnitude of this impact is determined by how much the actual value deviates from the model’s defined range.

Figure 11 provides valuable insights into the decision-making process of the proposed model by showcasing the features considered for each class before making a prediction. The figure includes five horizontal bar graphs, each representing one class, highlighting the top eight most impactful features for that class. These bar graphs indicate whether each feature had a positive or negative impact on the decision. The length and direction of the bars reveal how much each feature influenced the prediction.

A table displaying the prediction probabilities for each class is also included in Figure 11; the class with the highest probability is the final prediction. This prediction probability provides an indication of how each class is assessed and judged by the model. To further aid in comprehending the model’s decision-making process, the image additionally contains a table that lists the actual values of the characteristics for a specific test case inside the predicted class.

Figure 11 provides a detailed explanation of why the test case was predicted as a TDMA-type attack. The horizontal bar graphs for each class offer insights into the feature impacts that led to this prediction.

Starting with the Blackhole class, the first graph shows that the features have a nearly balanced impact, with both positive and negative influences, making it difficult for the model to decisively predict this class. Similarly, the graphs for the Normal and Grayhole classes display a mix of feature impacts, but these classes lean towards the negative side, making them less likely to be selected as the prediction.

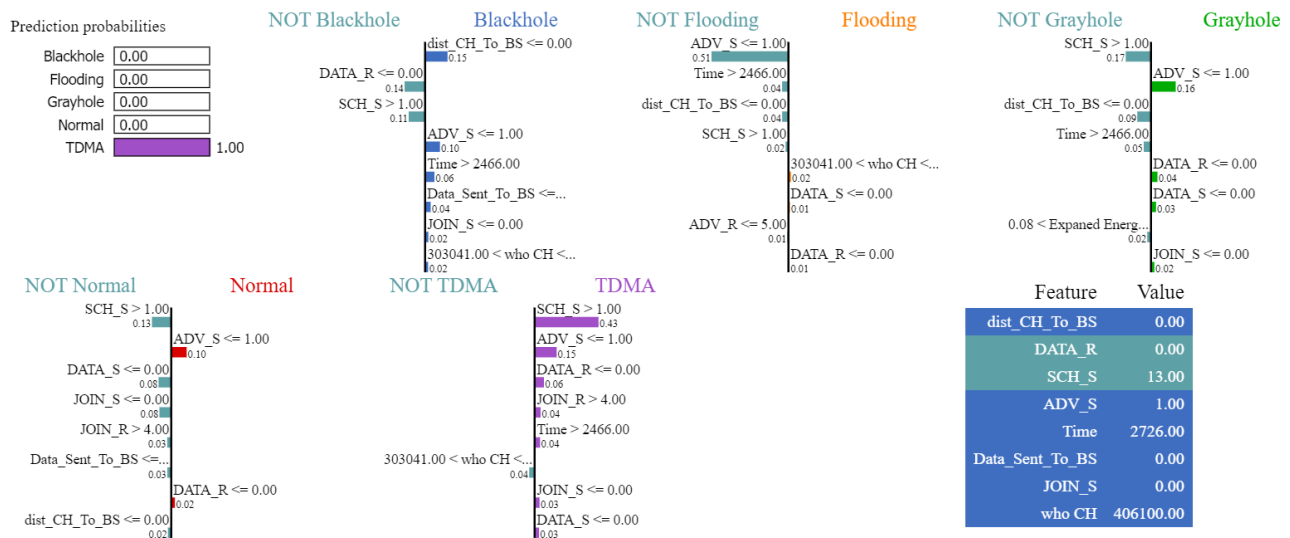


Figure 11. LIME tabular explainer for multiclass classification.



Figure 12. SHAP force plot for a particular test set (local).

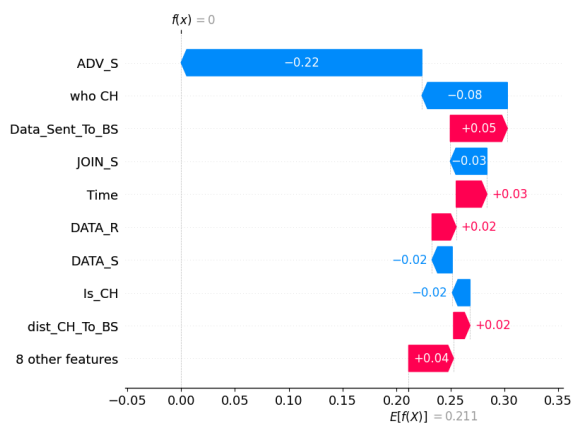


Figure 13. SHAP waterfall plot for a particular test set (local).

In contrast, the Flooding and TDMA classes each have a dominant feature that strongly influences the prediction. For the Flooding class, the feature “ADV_S” shows a significant 51% negative impact, which makes it straightforward for the model to rule out Flooding as the attack type. On the other hand, the TDMA class has the feature “SCH_S” with a 43% positive impact, which stands out as the most influential feature among all classes. This strong positive impact of

“SCH_S” makes TDMA the most likely prediction for the test case, according to the model’s decision-making process.

2) SHAP PLOT FOR A PARTICULAR TEST SET (LOCAL)

Among the most well-liked and simple-to-use tools for local representation in model interpretability are the SHAP force and waterfall plots. The contribution of each feature to a single ML model prediction is visually explained by the SHAP force diagram. It draws attention to the base value, which is usually displayed in the center of the plot and represents the predicted forecast prior to the consideration of any data. The anticipated value is displayed on the plot once feature contributions have been taken into consideration. The contribution of each attribute is shown as an arrow: blue arrows indicate negative impacts that drive the prediction below the base value, while red arrows indicate positive influences that push the prediction above it. These arrows’ lengths indicate the relative importance of each feature, giving a clear, sequential explanation of how the model arrived at its conclusion.

The SHAP waterfall plot conveys the same information as the SHAP force plot but does so in a two-dimensional format. In this plot, $E[f(x)]$ represents the base value or the average predicted value, serving as the starting point. The $f(x)$ indicates the final predicted value after considering

the contributions of each feature. The waterfall plot shows how each feature's contribution, either favorably or negatively, shifts the forecast away from the base value until it reaches the final prediction made by the model. It is simple to comprehend the cumulative effect of individual factors on the forecast thanks to this step-by-step graphic analysis.

Figures 12 and 13, which respectively represent the SHAP force plot and SHAP waterfall plot, illustrate the contribution of each feature in predicting the final outcome. The base value, or the average predicted value, is $E[f(x)] = 0.2113$, meaning that initially, without the influence of any features, the class had a 21.13% chance of being predicted as the outcome. The final prediction was $f(x) = 0$, indicating that after considering the influence of all features, the class has a 0% chance of being predicted as the outcome. In this case, the plots were designed to determine whether the network is under any kind of attack or not, focusing on the normal class in a multiclass prediction. The result, where $f(x) = 0$, means the test set has a 0% chance of being normal, confirming it is indeed an attack scenario. The SHAP plots also reveal the features and the magnitude of their influence on the outcome. For instance, the feature "Data_Sent_To_BS" provided the most positive influence, pushing the prediction probability higher, while the feature "ADV_S" had the most negative influence, lowering the prediction probability to the point where it became certain that the scenario was an attack.

3) SHAP SUMMARY PLOT FOR IMPORTANCE OF FEATURES (GLOBAL)

The SHAP summary plot is a visualization tool that shows how different attributes impact an ML model's output. In this two-dimensional graphic, the vertical and horizontal axes, respectively, reflect the feature names and the influence of the SHAP values. The SHAP values along the horizontal axis indicate each feature's impact on the model's output. A centerline divides these SHAP values; values to the right of the centerline suggest that the feature increases the expected value, while values to the left suggest that it decreases it. The titles of the features are positioned in descending order of influence along the vertical axis, with the most important features at the top. The plot also uses color to indicate feature value, with red indicating greater importance and blue indicating less relevance. It is simpler to see which features have the greatest influence on the model's predictions thanks to the color coding.

Figure 14 represents the SHAP summary plot for the Normal class of a test dataset. The plot reveals that the feature "dist_CH_To_BS" is the most important feature for the Normal class, as evidenced by its widest spread of SHAP values, indicating its significant impact on the model's outcome for that class. The feature also shows that most of the higher SHAP values are on the left side of the centerline, suggesting that as the value of "dist_CH_To_BS" increases, it tends to have a negative impact on the prediction. Other important features include "DATA_R," "ADV_S," and "SCH_S." The plot indicates that a low value of

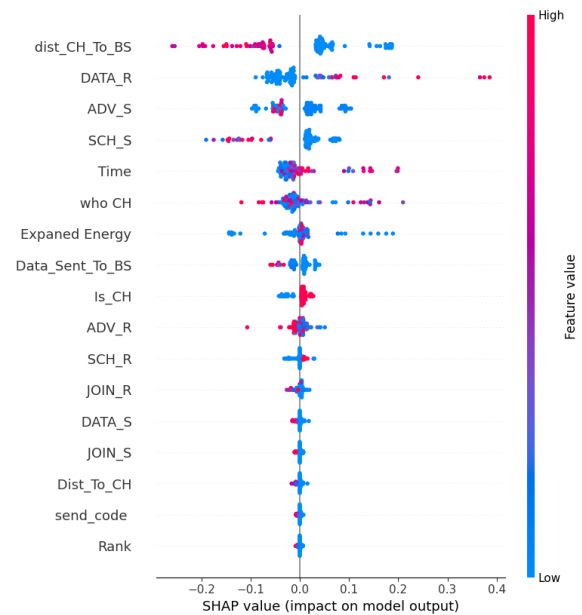


Figure 14. SHAP summary plot for normal class.

"dist_CH_To_BS," a high value of "DATA_R," and low values of "ADV_S" and "SCH_S" are associated with a normal, non-attack scenario.

4) SHAP SUMMARY PLOT OF THE VIOLIN DISTRIBUTION FOR IMPORTANCE OF FEATURE (GLOBAL)

The influence and density of features in the output of an ML model are shown by the SHAP summary plot with violin distribution. It has a horizontal and vertical axis and is a two-dimensional plot, just like the typical SHAP summary plot. The vertical axis provides a list of feature names, and the horizontal axis displays the SHAP values, which show how each feature affects the model's prediction. However, instead of showing the exact position of the SHAP values, the violin plot displays the density of these values at different positions, offering a visual representation of how densely the SHAP values are concentrated in certain areas. This helps to understand not just the magnitude of a feature's impact, but also how consistently that impact occurs across different predictions.

The violin representation of the SHAP values for the Normal class is shown in Figure 15, which also illustrates how different features affect the model's predictions. As the most important element in categorizing the Normal class, the characteristic "dist_CH_To_BS" stands out, with most greater SHAP values falling into the negative influence category. On the other hand, lower SHAP values improve the forecast. The density of SHAP values is the widest on the right side of the centerline, indicating a concentration of values that positively influence the prediction, though their proximity to the centerline suggests a minimal impact. Features like "ADV_S" and "SCH_S" show a similar level of influence, whereas "Expanded Energy" exhibits a more

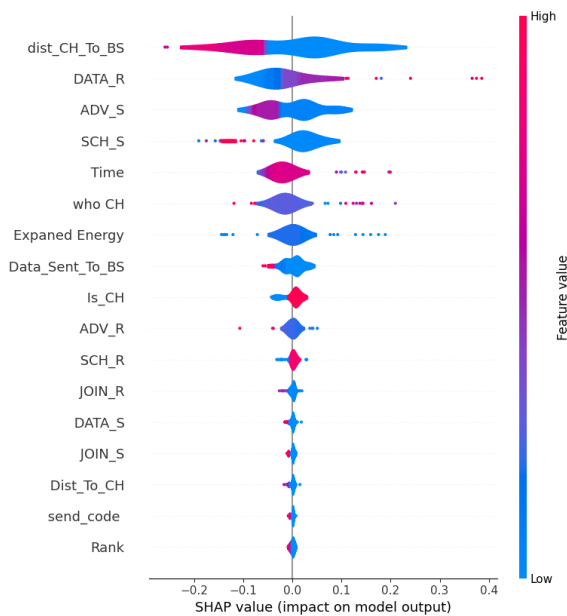


Figure 15. SHAP summary plot of the violin distribution.

even distribution of SHAP values. The predominance of blue in the plot indicates that these values are generally low, and since they are clustered near the centerline, it implies that this feature has a relatively minor impact on the prediction.

5) SHAP STACKED FORCE PLOT OF THE TEST SET (GLOBAL)

Supervised clustering of force plots, also known as stacked force plots, is a technique for understanding how various characteristics affect a model's predictions and for visualizing patterns and trends in the model's output. The vertical axis of this kind of plot shows the cumulative effect of characteristics on the decisions made by the model, while the horizontal axis represents the test samples. This kind of plot is a two-dimensional representation. You may detect trends and the relative importance of factors in influencing the model's results by stacking the force plots to see how specific features consistently affect predictions across numerous samples.

Figure 16 illustrates the output probability versus the test samples for 200 test instances. The test samples are represented by the horizontal axis, and the output probability is displayed by the vertical axis. From the plot, a clear dominance of the feature labeled "ADV_S" is evident. Initially, this feature had a slightly negative influence on the output, but as the samples progress, the feature increasingly shifts towards a significant positive influence, strongly affecting the model's final predictions.

The effects of the feature named "ADV_S" can be explicitly understood from Figure 17. The plot shows that while the feature had a maximum negative impact on the output, its impact was consistent throughout the test samples. This consistency highlights the importance of "ADV_S" as a critical feature for the model's prediction capabilities, even though its influence was primarily negative.

The results from both LIME and SHAP demonstrate a clear alignment in the identification of key features that dominate the decision-making process of the model. By examining the top four features highlighted in the SHAP summary plot and the LIME tabular plot, it becomes evident that these features consistently play a significant role in the model's predictions across various cases. This consistency across both methods enhances the reliability of the model's explanations, as the independent validation from each technique corroborates the importance and impact of these critical features on the model's outcomes.

G. CHECKING PREDICTIONS USING WEB APPLICATION

For ML and data science projects, Streamlit is an open-source Python framework that facilitates the development and dissemination of unique online applications. It makes it easier to convert data scripts into interactive applications with less coding work. A web application developed on Streamlit was used in this research project to verify the model's predictions. Figure 18 for normal scenarios and figures 19, 20, 21 & 22 for attacking scenarios demonstrate the consistency of the model's predictions across different attack types and normal conditions.

H. STRENGTH, LIMITATION, AND FUTURE WORK

The model developed in this research demonstrates consistently high accuracy, supported by extensive refinement and validation through a Streamlit web application. This performance highlights its effectiveness in identifying various attack types and normal conditions within the dataset. A key strength of our ensemble model is its high accuracy, achieved through PSO-tuned hyperparameters that outperform existing models. The integration of XAI techniques, such as SHAP and LIME, enhances interpretability, making the model practical for real-world applications by offering insights into its decision-making process. Additionally, our approach effectively addresses dataset imbalance and reduces false positives, allowing for scalability in large networks with diverse traffic patterns. This is particularly beneficial in critical domains like military surveillance, healthcare monitoring, and industrial automation, where real-time and accurate detection of security threats is essential. The application of SMOTE-Tomek for dataset balancing ensures better generalization across varying network conditions. Despite these strengths, the model has limitations primarily related to computational complexity. The ensemble of machine learning algorithms and optimization techniques increases computational overhead, potentially limiting real-time applicability in resource-constrained environments. Furthermore, the model's performance has been evaluated on specific datasets, and its generalization to other domains remains to be fully explored. The model has not been tested on dynamically changing datasets, where it may require different implementation approaches. Since the model has not been tested in real-life scenarios, its adaptability to various real-world situations remains uncertain. Future work should

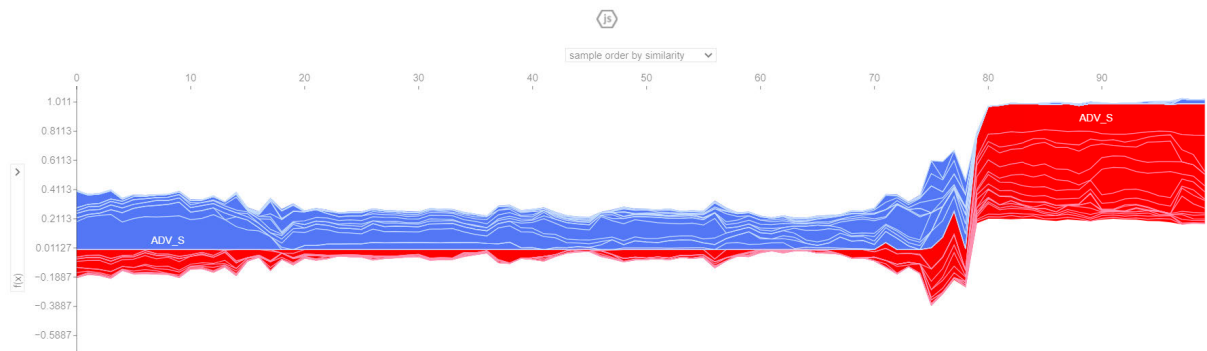


Figure 16. SHAP stacked force plot of the test sample based on sample order by similarity.

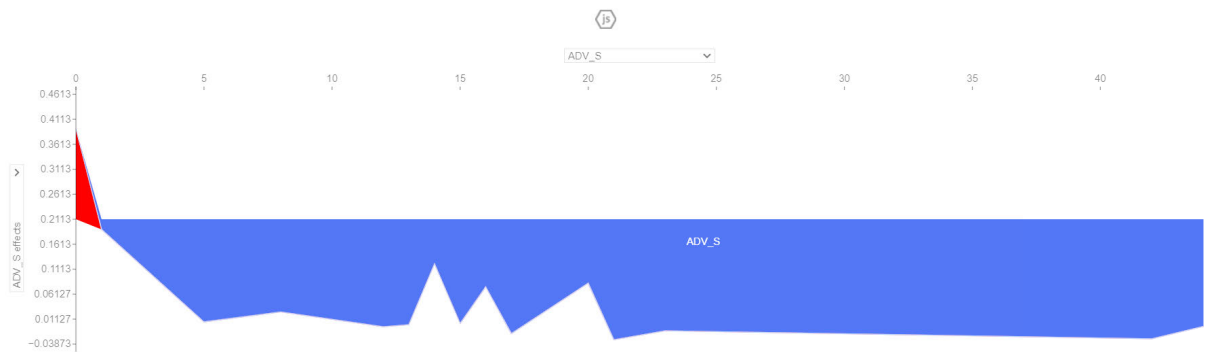


Figure 17. SHAP stacked force plot of the test sample analysing the feature "ADV_S".

Evaluation Of WSN attacks

ENTER DATA BELOW

Time	Is_CH	who_CH
200.00000000	0.00000000	104077.00000000
Dist_to_CH	ADV_S	ADV_R
12.06039000	0.00000000	2.00000000
JOIN_S	JOIN_R	SCH_S
1.00000000	0.00000000	0.00000000
SCH_R	Rank	DATA_S
1.00000000	6.00000000	20.00000000
DATA_R	DATA_Sent_to_BS	dist_CH_to_BS
0.00000000	0.00000000	0.00000000
send_code	Expanded Energy	
1.00000000	0.03424000	

Predict

The system Normal. Nothing to worry about.

About

Figure 18. Normal prediction in the web app.

Evaluation Of WSN attacks

ENTER DATA BELOW

Time	Is_CH	who_CH
53.00000000	1.00000000	101043.00000000
Dist_to_CH	ADV_S	ADV_R
0.00000000	1.00000000	0.00000000
JOIN_S	JOIN_R	SCH_S
0.00000000	27.00000000	1.00000000
SCH_R	Rank	DATA_S
0.00000000	0.00000000	0.00000000
DATA_R	DATA_Sent_to_BS	dist_CH_to_BS
1215.00000000	0.00000000	0.00000000
send_code	Expanded Energy	
0.00000000	0.00228000	

Predict

The system is under a Blackhole type attack.

About

Figure 19. Blackhole prediction in the web app.

focus on optimizing computational efficiency to enhance real-time applicability in resource-limited WSNs. Although this model has proven to be highly effective, the introduction

of newer techniques could further enhance its performance. Additionally, efforts should be made to make the model more lightweight and reduce prediction time to ensure

Evaluation Of WSN attacks

ENTER DATA BELOW

Time	Is_CH	who_CH
953.00000000	1.00000000	702016.00000000
Dist_to_CH	ADV_S	ADV_R
0.00000000	30.00000000	28.00000000
JOIN_S	JOIN_R	SCH_S
0.00000000	0.00000000	0.00000000
SCH_R	Rank	DATA_S
0.00000000	0.00000000	0.00000000
DATA_R	DATA_Sent_to_BS	dist_CH_to_BS
0.00000000	13.00000000	54.93620000
send_code	Expanded Energy	
0.00000000	0.22030000	

Predict

The system is Flooding.

About

Figure 20. Flooding prediction in the web app.

Evaluation Of WSN attacks

ENTER DATA BELOW

Time	Is_CH	who_CH
853.00000000	1.00000000	117028.00000000
Dist_to_CH	ADV_S	ADV_R
0.00000000	1.00000000	2.00000000
JOIN_S	JOIN_R	SCH_S
0.00000000	53.00000000	53.00000000
SCH_R	Rank	DATA_S
0.00000000	0.00000000	0.00000000
DATA_R	DATA_Sent_to_BS	dist_CH_to_BS
24.00000000	0.00000000	0.00000000
send_code	Expanded Energy	
0.00000000	0.00725000	

Predict

The system is under a TDMA type attack.

About

Figure 22. TDMA prediction in the web app.

Evaluation Of WSN attacks

ENTER DATA BELOW

Time	Is_CH	who_CH
53.00000000	1.00000000	101029.00000000
Dist_to_CH	ADV_S	ADV_R
0.00000000	1.00000000	0.00000000
JOIN_S	JOIN_R	SCH_S
0.00000000	30.00000000	1.00000000
SCH_R	Rank	DATA_S
0.00000000	0.00000000	0.00000000
DATA_R	DATA_Sent_to_BS	dist_CH_to_BS
1230.00000000	22.00000000	155.90190000
send_code	Expanded Energy	
0.00000000	2.27972000	

Predict

The system is under a Grayhole type attack.

About

Figure 21. Grayhole prediction in the web app.

improved performance when deployed on edge devices. Expanding testing on larger and more diverse datasets will be crucial for ensuring robustness and ensure the fact the the model can be implemented in every possible scenarios. Furthermore, integrating adaptive learning mechanisms will enable the model to evolve alongside emerging attack vectors, ensuring its continued relevance and effectiveness in dynamic environments.

V. CONCLUSION

This research presents a highly effective and scalable IDS for WSN, combining cutting-edge ML techniques and optimization strategies. The proposed system, leveraging PSO and an ensemble of Random Forest, Decision Tree, and K-Nearest Neighbors models, achieved an exceptional accuracy of 99.73%, with precision, recall, and F1-score all reaching 99.72%. The use of advanced data balancing methods like SMOTE-Tomek significantly improved the detection of minority attack instances, overcoming the challenges posed by dataset imbalance. Additionally, explainable AI techniques such as SHAP and LIME provided crucial insights into the model's decision-making process, enhancing transparency and interpretability, key for practical applications. Moreover, the integration of the Streamlit web application allowed for an intuitive and interactive interface, enabling real-time validation of the model's predictions across different attack types and normal conditions, further demonstrating the model's robustness and user accessibility. Future work should focus on optimizing the system's computational efficiency for deployment in resource-constrained environments and extending its applicability to broader, more diverse datasets to ensure greater generalizability across different WSN scenarios.

ACKNOWLEDGMENT

Qatar National Library (QNL) will cover the open access publication cost.

References

- [1] S. Salmi and L. Oughdir, "Performance evaluation of deep learning techniques for DoS attacks detection in wireless sensor network," *J. Big Data*, vol. 10, no. 1, p. 17, Feb. 2023.

- [2] X. Feng, X. Ding, and S. Sun, "A security detection scheme based on evidence nodes in wireless sensor networks," in *Proc. 6th Int. Conf. Biomed. Eng. Informat.*, Dec. 2013, pp. 689–693.
- [3] M. N. U. Islam, A. Fahmin, M. S. Hossain, and M. Atiquzzaman, "Denial-of-service attacks on wireless sensor network and defense techniques," *Wireless Pers. Commun.*, vol. 116, no. 3, pp. 1993–2021, Feb. 2021.
- [4] M. A. Hasan, F. Haque, S. R. Sabuj, H. Sarker, M. O. F. Goni, F. Rahman, and M. M. Rashid, "An end-to-end lightweight multi-scale CNN for the classification of lung and colon cancer with XAI integration," *Technologies*, vol. 12, no. 4, p. 56, Apr. 2024.
- [5] H. S. Sharma, M. M. Singh, and A. Sarkar, "Machine learning-based DoS attack detection techniques in wireless sensor network: A review," in *Proc. Int. Conf. Cognit. Intell. Comput.*, vol. 2. Singapore: Springer, Jan. 2023, pp. 583–591.
- [6] M. Dener, S. Al, and A. Orman, "STLGBM-DDS: An efficient data balanced DoS detection system for wireless sensor networks on big data environment," *IEEE Access*, vol. 10, pp. 92931–92945, 2022.
- [7] S. Jiang, J. Zhao, and X. Xu, "SLGBM: An intrusion detection mechanism for wireless sensor networks in smart environments," *IEEE Access*, vol. 8, pp. 169548–169558, 2020.
- [8] I. A. Khan, M. Keshk, D. Pi, N. Khan, Y. Hussain, and H. Soliman, "Enhancing IIoT networks protection: A robust security model for attack detection in internet industrial control systems," *Ad Hoc Netw.*, vol. 134, Sep. 2022, Art. no. 102930.
- [9] S. Sharmin, I. Ahmedy, and R. M. Noor, "An energy-efficient data aggregation clustering algorithm for wireless sensor networks using hybrid PSO," *Energies*, vol. 16, no. 5, p. 2487, Mar. 2023.
- [10] N. M. Alruhaily and D. M. Ibrahim, "A multi-layer machine learning-based intrusion detection system for wireless sensor networks," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 4, pp. 281–288, 2021.
- [11] A. G. Putrada, N. Alamsyah, S. F. Pane, and M. N. Fauzan, "XGBoost for IDS on WSN cyber attacks with imbalanced data," in *Proc. Int. Symp. Electron. Smart Devices (ISESD)*, Nov. 2022, pp. 1–7.
- [12] N. Dharini, J. Katiravan, D. M. S. Priya, and S. V. A. Sakthi, "Intrusion detection in novel WSN-leach dos attack dataset using machine learning based boosting algorithms," *Proc. Comput. Sci.*, vol. 230, pp. 90–99, Jan. 2023.
- [13] X. Tan, S. Su, Z. Huang, X. Guo, Z. Zuo, X. Sun, and L. Li, "Wireless sensor networks intrusion detection based on SMOTE and the random forest algorithm," *Sensors*, vol. 19, no. 1, p. 203, Jan. 2019.
- [14] A. B. Abhale and A. J. Reddy, "Deep learning perspectives to detecting intrusions in wireless sensor networks," *Int. J. Intell. Syst. Appl. Eng.*, vol. 11, no. 2, pp. 18–26, Jan. 2023.
- [15] D. Hemanand, G. Reddy, S. S. Babu, K. R. Balmuri, T. Chitra, and S. Gopalakrishnan, "An intelligent intrusion detection and classification system using CSGO-LSVM model for wireless sensor networks (WSNs)," *Int. J. Intell. Syst. Appl. Eng.*, vol. 10, no. 3, pp. 285–293, Oct. 2022.
- [16] P. R. Chandre, P. Mahalle, and G. Shinde, "Intrusion prevention system using convolutional neural network for wireless sensor network," *IAES Int. J. Artif. Intell. (IJ-AI)*, vol. 11, no. 2, p. 504, Jun. 2022.
- [17] N. Singh, D. Virmani, and X.-Z. Gao, "A fuzzy logic-based method to avert intrusions in wireless sensor networks using WSN-DS dataset," *Int. J. Comput. Intell. Appl.*, vol. 19, no. 3, Sep. 2020, Art. no. 2050018.
- [18] V. Gowdhaman and R. Dhanapal, "An intrusion detection system for wireless sensor networks using deep neural network," *Soft Comput.*, vol. 26, no. 23, pp. 13059–13067, Dec. 2022.
- [19] M. H. Behiry and M. Aly, "Cyberattack detection in wireless sensor networks using a hybrid feature reduction technique with AI and machine learning methods," *J. Big Data*, vol. 11, no. 1, p. 16, Jan. 2024.
- [20] G. G. Gebremariam, J. Panda, and S. Indu, "Design of advanced intrusion detection systems based on hybrid machine learning techniques in hierarchically wireless sensor networks," *Connection Sci.*, vol. 35, no. 1, Dec. 2023, Art. no. 2246703.
- [21] M. A. Elsadig, "Detection of denial-of-service attack in wireless sensor networks: A lightweight machine learning approach," *IEEE Access*, vol. 11, pp. 83537–83552, 2023.
- [22] M. Karthikeyan, D. Manimegalai, and K. RajaGopal, "Firefly algorithm based WSN-IoT security enhancement with machine learning for intrusion detection," *Sci. Rep.*, vol. 14, no. 1, p. 231, Jan. 2024.
- [23] G. Liu, H. Zhao, F. Fan, G. Liu, Q. Xu, and S. Nazir, "An enhanced intrusion detection model based on improved kNN in WSNs," *Sensors*, vol. 22, no. 4, p. 1407, Feb. 2022.
- [24] M. Aljebreen, M. A. Alohal, M. K. Saeed, H. Mohsen, M. Al Duhayyim, A. A. Abdelmageed, S. Drar, and S. Abdelbagi, "Binary chimp optimization algorithm with ML based intrusion detection for secure IoT-assisted wireless sensor networks," *Sensors*, vol. 23, no. 8, p. 4073, Apr. 2023.
- [25] T. M. Nguyen, H. H.-P. Vo, and M. Yoo, "Enhancing intrusion detection in wireless sensor networks using a GSWO-CatBoost approach," *Sensors*, vol. 24, no. 11, p. 3339, May 2024.
- [26] R. Bocu, M. Iavich, and S. Tabirca, "A real-time intrusion detection system for software defined 5G networks," in *Proc. Int. Conf. Advanced Inf. Netw. Appl. Cham, Switzerland: Springer*, May 2021, pp. 436–446.
- [27] N. Marriwala and P. Rathee, "An approach to increase the wireless sensor network lifetime," in *Proc. World Congr. Inf. Commun. Technol.*, Oct. 2012, pp. 495–499.
- [28] D. Shah, Z. Yu Xue, and T. M. Aamodt, "Label encoding for regression networks," 2022, *arXiv:2212.01927*.
- [29] F. Aldi, F. Hadi, N. A. Rahmi, and S. Defit, "Standardscaler's potential in enhancing breast cancer accuracy using machine learning," *J. Appl. Eng. Technological Sci. (JAETS)*, vol. 5, no. 1, pp. 401–413, Dec. 2023.
- [30] A. Fernandez, S. Garcia, F. Herrera, and N. V. Chawla, "SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary," *J. Artif. Intell. Res.*, vol. 61, pp. 863–905, Apr. 2018.
- [31] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [32] R. M. Pereira, Y. M. G. Costa, and C. N. Silla Jr., "MLTL: A multi-label approach for the totem link undersampling algorithm," *Neurocomputing*, vol. 383, pp. 95–105, Mar. 2020.
- [33] M. A. Talukder, S. Sharmin, M. A. Uddin, M. M. Islam, and S. Aryal, "MLSTL-WSN: Machine learning-based intrusion detection using SMOTETomek in WSNs," *Int. J. Inf. Secur.*, vol. 23, no. 3, pp. 2139–2158, Jun. 2024.
- [34] S. Hart, "Shapley value," in *Game Theory*. London, U.K.: Palgrave Macmillan, 1989, pp. 210–216.
- [35] J. Deegan and E. W. Packel, "A new index of power for simple n-person games," *Int. J. Game Theory*, vol. 7, pp. 113–123, Jun. 1978.
- [36] G. Visani, E. Bagli, F. Chesani, A. Poluzzi, and D. Capuzzo, "Statistical stability indices for LIME: Obtaining reliable explanations for machine learning models," *J. Oper. Res. Soc.*, vol. 73, no. 1, pp. 91–101, Jan. 2022.
- [37] P. Singh, "Machine learning deployment as a web service," in *Deploy Machine Learning Models to Production: With Flask, Streamlit, Docker, and Kubernetes on Google Cloud Platform*. Springer, 2021, pp. 67–90.
- [38] D. Jeevaraj, B. Karthik, M. Sriram, S. P. Vijayaragavan, and D. Gokulakrishnan, "Intrusion detection in WSN using supervised machine learning techniques," *Int. J. Intell. Syst. Appl. Eng.*, vol. 12, no. 9, pp. 483–490, 2023.
- [39] D. Jeevaraj, B. Karthik, T. Vijayan, and M. Sriram, "Feature selection model using naive Bayes ML algorithm for WSN intrusion detection system," *Int. J. Electr. Comput. Eng. Syst.*, vol. 14, no. 2, pp. 179–185, Feb. 2023.
- [40] M. Khorasani, M. Abdou, and J. H. Fernández, "Web application development with streamlit," in *Software Development*. Springer, 2022, pp. 498–507.
- [41] I. Almomani, B. Al-Kasasbeh, and M. Al-Akhras, "WSN-DS: A dataset for intrusion detection systems in wireless sensor networks," *J. Sensors*, vol. 2016, Jan. 2016, Art. no. 4731953.



SHAIKH AFNAN BIRAHIM is currently pursuing the M.Sc. degree in computing science with the University of Glasgow. He is a passionate Data Scientist and a Computing Enthusiast, he is actively exploring the fields of machine learning and AI, developing innovative models, and conducting research in applied deep learning, interpretability, and computer vision. With experience as a Software Engineer, he is also a Lecturer with the Royal University of Dhaka, where he is on study leave.

He is expanding his expertise to drive innovation and shape the future of technology in the era of the fourth industrial revolution with the University of Glasgow.



AVIJIT PAUL is currently pursuing the bachelor's degree with the Department of Electronics and Telecommunication Engineering, Rajshahi University of Engineering and Technology. His research interests include machine learning, deep learning, and artificial intelligence, where he actively seeks to deepen his knowledge and expertise. He is committed to leveraging these technologies to solve real-world problems and contribute to advancements in the field.



MOHAMMAD ASIF HASAN received the bachelor's degree in electronics and telecommunication engineering from Rajshahi University of Engineering and Technology. His research interests include the intersection of healthcare, agriculture, and technology, with a focus on machine learning and deep learning, with a passion for computer vision, medical and agriculture image processing, medical and eeg signal processing, and medical and agriculture data analysis. He has consistently demonstrated a commitment to academic excellence. He has engaged in several research projects, showcasing their strong analytical skills and collaborative ability. With a commitment to staying up-to-date with the latest developments in the field, he is poised to make a meaningful contribution to the future of healthcare and agriculture through technology.



FAHMIDA RAHMAN (Member, IEEE) is currently an artificial intelligence enthusiast deeply fascinated by the immense possibilities this field offers. With a strong drive to explore the depths of AI, she is committed to gaining a thorough understanding of the core concepts, from machine learning and neural networks to natural language processing and computer vision. Her journey is fueled by a passion for problem-solving and innovation, where she constantly seeks to learn

new methodologies, experiment with cutting-edge technologies, and uncover groundbreaking applications of AI.



FARIHA HAQUE received the bachelor's degree in electronics and telecommunication engineering from Rajshahi University of Engineering and Technology. Her research interests include the convergence of healthcare, agriculture, and technology, with a particular focus on machine learning and deep learning. She is deeply passionate about computer vision, medical and agricultural image processing, and data analysis. She has actively contributed to several research projects throughout her research journey, showcasing her strong analytical abilities and collaborative approach. Committed to staying at the forefront of AI advancements, she aims to drive innovative solutions that will shape the future of healthcare and agriculture.



YAMINA ISLAM received the B.Sc. degree in computer science and engineering (CSE) from IIUC. She is currently pursuing the M.Sc. degree in CSE with Chittagong University of Engineering and Technology (CUET). She is an Assistant Lecturer with International Islamic University Chittagong (IIUC), Bangladesh. Her research interests include artificial intelligence, machine learning, deep learning, and natural language processing. She has also conducted research on

women's security, particularly in the field of human activity detection. Her current research interests include advancing AI-driven solutions to enhance security and safety through activity detection and related technologies.



MUHAMMAD E. H. CHOWDHURY (Senior Member, IEEE) received the Ph.D. degree from the University of Nottingham, U.K., in 2014. He subsequently worked as a Postdoctoral Research Fellow with the Sir Peter Mansfield Imaging Centre, University of Nottingham. He is currently an Assistant Professor and a Program Coordinator with the Department of Electrical Engineering, Qatar University. He is a prolific Researcher with several patents to his name, three edited books, and

over 225 peer-reviewed journal articles, more than 30 conference papers, and multiple book chapters. He is actively involved in leading several research projects funded by Qatar Research, Development, and Innovation Council (QRDI) and internal grants from Qatar University, along with academic collaborations with HBKU and HMC. His research interests include biomedical instrumentation, signal processing, wearable sensors, medical image analysis, machine learning, computer vision, embedded system design, and simultaneous EEG/fMRI. He has been recognized with several prestigious awards, including the COVID-19 Dataset Award, the AHS Award from HMC, and the National AI Competition Award for his contributions to the fight against COVID-19. His team earned a gold medal at the 13th International Invention Fair in the Middle East (IIFME). Additionally, he has been listed among the Top 2% of scientists in the world by Stanford University. He also serves as an Associate Editor for *Computers and Electrical Engineering* and *IEEE Access* and the Topic Editor and the Review Editor for *Frontiers in Neuroscience*.

...



TONMOY ROY is currently pursuing the degree with Utah State University, USA, focusing on the application of machine learning in computing. His research interests include machine learning, deep learning, the Internet of Things (IoT), natural language processing, artificial intelligence (AI), and computer vision. Driven by a passion for research, he actively participates in collaborative projects that seek to drive technological innovation and advancement.